

Student Productivity Tracker

Project Documentation

TEAM SPRINSTERS

2025

BMS Institute of Technology and Management

Department of Artificial Intelligence and Machine Learning

Submitted by:

Kunal

Atul Dhull

Ritika Girish Kulkarni

Abstract

The **Student Productivity Tracker** is a web and Android-based system developed to help students convert academic workload into a realistic, data-driven plan. The application combines task management, routine planning, health tracking and analytics into a single interface. A central dashboard presents completion status, planned study hours, subject-wise focus and a shortlist of “Today’s Missions”. A separate health hub records sleep, screen time, activity, hydration, meals and mood to emphasise balanced productivity rather than long unbroken study hours.

The project is implemented using React and a Node.js backend, with data stored locally in the browser and persisted through backend APIs. Capacitor is used to package the web app as a native Android application. Push notifications are enabled using web push with VAPID keys. An integrated AI assistant reads data from the application through APIs and is able to answer questions about the user’s timetable and progress, as well as provide short recommendations and motivational prompts.

This documentation explains the system architecture with particular focus on four aspects required in the assignment: *AI logic*, *routine flow*, *task scheduling logic* and the *database schema*. Additional sections describe the health and well-being module, dashboard analytics, assistant and API integrations, software requirements, Android deployment process, testing and future enhancements.

Acknowledgment

The completion of this project has been supported by guidance from the faculty of the Department of Artificial Intelligence and Machine Learning at **BMS Institute of Technology and Management**. Suggestions on interface layout, modular design and documentation structure contributed significantly to the final outcome.

Feedback from fellow students during testing also shaped several features, particularly the health hub layout, daily missions section and routine configuration options. Encouragement from family members throughout the development and debugging stages is gratefully acknowledged.

Contents

Abstract	1
Acknowledgment	2
1 Overview	4
2 Software Requirements	6
3 Documentation	7
3.1 AI Logic	7
3.2 Routine Flow	9
3.3 Task Scheduling Logic	10
3.4 Database Schema	11
4 Health and Well-being Module	14
5 AI Assistant and API Integration	15
6 Android Deployment	17
7 Testing and Results	18
8 Future Enhancements	19
9 Conclusion	20

1 Overview

The Student Productivity Tracker is designed for students who are aware of their academic responsibilities but struggle to translate them into a consistent daily routine. Assignments, quizzes and projects are often known well in advance, yet work tends to accumulate near deadlines due to lack of planning or feedback about how time is being used.

The application addresses this challenge by combining four pillars:

1. **Tasks and routines** for capturing what needs to be done and when.
2. **Analytics and AI logic** for interpreting activity into signals such as productivity scores, focus patterns and mission suggestions.
3. **Health tracking** for monitoring sleep, screen time, activity, hydration, meals and mood to support sustainable study habits.
4. **An AI assistant** that can read the stored data and answer questions about the timetable, pending work and overall balance.

The main dashboard greets the user with a themed header and three summary cards:

- **Tasks Completed** — shows completed and pending tasks and a productivity percentage.
- **Study Plan** — displays the number of scheduled study sessions and total hours planned through the Routine page.
- **Subjects Mastered** — summarises progress across subjects or modules.

Below these cards, the dashboard provides a visual overview of academic activity: a conceptual representation of completed versus pending work, a chart of study time over recent days, a subject-wise focus display and a “Today’s Missions” list that surfaces a small set of high-priority tasks. A “Daily Buff” message delivers short motivational text tuned to recent usage patterns.

A separate **Cosmic Health Hub** page integrates health-related information with academic context. It includes dedicated sections for sleep tracking, screen time, daily activity, hydration and meals, mood and weekly health summary. The health overview at the top condenses average sleep, screen time, activity and water intake into simple bars and targets.

The system architecture is layered. React components form the presentation layer, React context and utility modules manage shared state and business logic, and a Node.js backend provides API endpoints for persistence and integration. Browser storage is used

as a fast local cache, while the backend stores structured records that can later be migrated to full cloud storage. The following sections provide detailed documentation of the internal logic.

2 Software Requirements

Hardware Requirements

- Development machine with at least 8 GB RAM and a multi-core processor.
- Around 10 GB of free disk space for Node modules, Android SDK and build artifacts.
- Android smartphone with USB debugging enabled for testing Capacitor builds.

Software Requirements

- **Operating System:** Windows 10 or later, or a compatible Linux distribution.
- **Node.js and npm:** for running the React dev server and building production bundles.
- **React:** front-end library used for the single-page application.
- **Capacitor:** for wrapping the web app into a native Android app.
- **Android Studio:** for building, running and debugging the Android project.
- **Node.js Backend:** Express-based server to store tasks, routines, health logs and to expose AI and timetable APIs.
- **Web Push stack:** VAPID key pair configured in `.env` files for backend and a corresponding public key exposed to the React app.
- **Git:** for version control.
- **Modern browser:** for web testing of the application interface.

The stack keeps the deployment path relatively simple while still reflecting common tools used in current web and mobile development.

3 Documentation

The assignment specifies four major documentation topics: AI logic, routine flow, task scheduling logic and database schema. These are described below in the context of the Student Productivity Tracker implementation.

3.1 AI Logic

In this project, AI logic refers to the analytical layer that converts raw usage data into meaningful suggestions and visualisations. The logic is implemented in utility modules (for example an `aiEngine` file) that operate over stored tasks, routines, health logs and analytic summaries obtained via backend APIs.

Input data sources

The AI logic consumes information from multiple sources:

- **Task events:** creation, update and completion of tasks, including timestamps and subject tags.
- **Routine schedule:** planned study blocks with durations and recurrence.
- **Health logs:** daily entries for sleep duration, screen time, activity minutes, water intake, meals, and mood.
- **Backend summaries:** pre-aggregated metrics returned by the Node.js backend, such as weekly averages.

These records are stored both in local storage (for quick access in the UI) and in backend storage via REST APIs. API keys and base URLs are kept in environment variables, ensuring that secrets such as `OPENROUTER_API_KEY`, `OCRSPACE_API_KEY` and web push keys are not exposed directly in the code.

Productivity calculations

The dashboard's *Tasks Completed* card displays a productivity percentage. For a chosen time window (for example, today or this week), tasks are partitioned into completed and pending. A typical computation is:

$$\text{Productivity\%} = \frac{\text{Completed_window}}{\text{Completed_window} + \text{Pending_window} + \varepsilon} \times 100,$$

where ε is a small constant to avoid division by zero when there is little data. Only tasks whose due dates are within or before the current window are counted so that long-term future tasks do not distort the metric.

Study time and focus analytics

Study time over days and subject-wise focus rely on aggregated durations. The backend or AI module groups events by day and subject. For a given day d ,

$$T(d) = \sum_{k \in S_d} \text{duration}(k),$$

where S_d is the set of sessions or completed tasks associated with day d . Similarly, for a subject s ,

$$F(s) = \sum_{k \in S_s} \text{duration}(k),$$

where S_s is the set of session or task records tagged with subject s . These values are passed into chart components on the dashboard.

Health-aware insights

The AI logic also uses health data to generate insights on the health hub. Average sleep, average screen time, total weekly activity and average water intake are computed from daily health logs. For example, average sleep over a week is

$$\text{AvgSleep} = \frac{1}{n} \sum_{d=1}^n \text{sleepHours}(d),$$

where n is the number of logged days. Similar expressions are used for screen time, water intake and activity minutes. These metrics are compared against recommended ranges and summarised in short textual insights such as “Screen time is within a healthy study-friendly range” or “Hydration is below target; more water intake is suggested.”

Interaction with the AI assistant

The integrated AI assistant accesses backend APIs to retrieve the user’s timetable, current task list and health summaries. When the user asks a question such as “What should be studied this evening?” or “How was focus this week?”, the assistant first fetches structured data (for example, upcoming tasks with deadlines, adherence to routines, health averages) and then composes a natural language answer. API keys for the assistant are configured in environment variables, and the backend acts as a mediator so that sensitive keys are not exposed to the client.

Typical questions answered include:

- status of pending tasks and urgent deadlines,
- recommended subjects to focus on based on past performance,

- simple reflections such as whether sleep and screen habits support the current study goals.

The AI logic is intentionally rule-based and data-driven rather than opaque, making it suitable for an academic project where explanation of behaviour is important.

3.2 Routine Flow

Routine flow covers how study sessions are defined, interpreted in real time and incorporated into analytics. The Routine page allows the user to create structured blocks such as “Web Development Practice” or “Math Revision” with start and end times and recurrence patterns.

Routine definition and storage

Each routine entry stores:

- a unique routine identifier,
- activity name,
- associated subject or tag,
- start time and end time,
- recurrence pattern (daily, weekday-only, or specific days),
- active flag.

These entries are saved locally and synchronised with the backend through API calls, ensuring that routines are preserved even if the application is reinstalled or accessed from another device in future versions.

Determining active and upcoming blocks

When the dashboard is loaded, the current time is compared with the routine schedule. The logic marks each block as:

- *upcoming* if its start time is later on the same day,
- *active* if the current time falls between start and end,
- *past* once it has ended.

Active blocks can influence the “Today’s Missions” list by favouring tasks matching the subject of the currently active routine. Upcoming blocks may be highlighted so that the user knows what is planned for later in the day.

Study plan summary

The Study Plan card on the dashboard uses routine data to calculate how many sessions and total hours are scheduled. The total duration H for a given horizon (for example, the current week) is computed by summing all active blocks within that horizon:

$$H = \sum_{r \in R} \text{duration}(r),$$

where R is the set of routine instances (including repeated occurrences due to recurrence patterns). The number of planned blocks is displayed alongside the total hours, giving a quick sense of whether the timetable is empty or overloaded.

Adherence and behavioural feedback

Adherence to routines is estimated by checking whether tasks related to a block's subject were completed or updated during the block's active time. Blocks with relevant activity are counted as honoured; those without are considered skipped. An adherence percentage over a week supports feedback such as “Evening sessions are followed consistently, but early morning blocks are often skipped,” which can be used to adjust future schedules.

3.3 Task Scheduling Logic

Task scheduling logic determines which tasks appear as “Today's Missions” and how other tasks are prioritised in lists. Each task record contains attributes such as id, title, subject, due date, estimated hours, difficulty and status.

Scoring model

A simple but effective scoring model is used. For each task t , a priority score is computed as:

$$\text{PriorityScore}(t) = U(t) + D(t) - L(t),$$

where:

- $U(t)$ represents urgency based on how close the due date is,
- $D(t)$ represents difficulty or importance, sometimes scaled by estimated hours,
- $L(t)$ represents a load penalty used to keep daily plans realistic.

Urgency is typically modelled by the function:

$$U(t) = \max(0, U_{\max} - \alpha \cdot \text{daysLeft}(t)),$$

where $\text{daysLeft}(t)$ is the number of days until the due date, $U_{\max}$ is a maximum urgency value and α is a decay constant.

Difficulty is derived from a scale (for example, easy = 1, medium = 2, hard = 3) and may be multiplied by estimated hours to reflect combined effort. The load penalty increases as more tasks are added to the missions list for a given day and aims to keep total planned hours within a sensible limit.

Selecting Today's Missions

The following steps are used to construct the missions list:

1. Filter tasks to those whose status is pending or in progress.
2. Discard tasks with due dates far beyond the chosen planning horizon.
3. Compute a priority score for each remaining task.
4. Sort the tasks in descending order of priority score.
5. Select the top tasks while the sum of their estimated hours stays under a configuration threshold (for example, 3–4 hours).

The selected tasks are displayed on the dashboard in the “Today’s Missions” panel with subject labels and estimated durations. Completion buttons allow quick updates; completed missions are removed from the list and the next tasks in the priority order move up into the visible set.

Impact on dashboard and assistant

The missions system ensures that the user is not confronted with a large unstructured task list each day. Instead, a small, high-impact subset is surfaced. The AI assistant also uses the same priority logic when answering questions such as “What should be done first today?” or “Which assignment is most urgent now?”.

3.4 Database Schema

The database schema is designed to support both local storage and backend persistence, with a clear path to future cloud migration. Four primary logical entities are used: profile, task, routine and health/analytics.

Profile entity

The profile entity contains information used for personalisation and settings:

- student name or display name,

- default focus area tagline (for example, “Focus: web”),
- chosen visual theme (e.g. Inferno Red, grid mode),
- notification preferences and integration flags.

Task entity

The task entity has fields such as:

- `taskId` (primary identifier),
- `title`, `description`,
- `subjectCode` or category,
- `createdDate`, `dueDate`,
- `estimatedHours`,
- `difficultyLevel`,
- `status` (pending, in-progress, completed, deferred).

In the backend, task records can be stored in a collection or table keyed by user id and task id, allowing queries such as “all pending tasks for this student” or “tasks due in the next 7 days”.

Routine entity

Routine records store:

- `routineId`,
- `activityName`,
- `subjectCode`,
- `startTime`, `endTime`,
- `recurrencePattern`,
- `isActive`.

Routine instances can be expanded into a schedule of concrete blocks for a given week when needed.

Health and analytics entities

The health module uses daily records representing:

- date,
- total sleep hours and qualitative sleep rating,
- screen time hours,
- activity minutes or steps,
- glasses of water, meals and fruit/vegetable servings,
- junk food indicator,
- mood description, stress rating and affirmation text.

In addition, separate analytics collections maintain per-day or per-week summaries such as average sleep, average screen time and productivity scores. These can be stored in the backend database to support long-term trends, while the frontend caches recent results for fast loading.

The current implementation stores data primarily using backend JSON stores or simple database tables with local storage acting as a cache. The schema has been designed so that it can be mapped to cloud databases with minimal change to business logic.

4 Health and Well-being Module

The health and well-being module, presented as the *Cosmic Health Hub*, recognises that academic performance is strongly influenced by sleep, screen habits, physical activity and emotional state. Instead of focusing only on tasks, the application allows students to log and review these factors.

The health hub contains several panels:

- **Sleep Tracking:** logs total hours slept, deep and light sleep components and subjective quality rating.
- **Screen Time:** records today's screen usage and a seven-day average, with a chart for recent days.
- **Daily Activity:** captures steps and active minutes, either manually or via integration with external activity data.
- **Hydration and Meals:** stores water intake, number of meals, servings of fruits and vegetables and junk food flags.
- **Mood and Mindspace:** allows tracking mood, stress level, notes and positive affirmations; the latest affirmation is highlighted.
- **Weekly Health Overview:** presents average sleep, screen time, activity and water intake along with simple recommendations.

The health module shares structural patterns with the rest of the system. Each log is represented as a record in storage, and the AI logic aggregates the logs into averages and trends that are displayed in both chart and text form. The quick insights section summarises these findings in a few bullet points so that the student can quickly understand whether their recent habits support or hinder productivity.

5 AI Assistant and API Integration

A key feature of the Student Productivity Tracker is an AI assistant that is aware of the user's data. The assistant is accessed from a floating button on the interface and is able to answer questions using information from tasks, routines, health logs and timetable APIs.

Backend APIs

The backend exposes endpoints supporting operations such as:

- retrieving tasks, routines and health statistics,
- updating completion status and logs,
- computing aggregated analytics,
- forwarding queries to external AI or timetable services using secure API keys.

Environment variables defined in `.env` files store keys such as `OPENROUTER_API_KEY` for conversational AI, `OCRSPACE_API_KEY` if timetable extraction from images is enabled, and `VAPID` keys for push notifications. On the frontend side, public keys required by service workers are exposed using variables like `REACT_APP_VAPID_PUBLIC_KEY`.

Assistant capabilities

When a user interacts with the assistant, the typical flow is:

1. The frontend sends the user's question to the backend along with an anonymised identifier.
2. The backend queries the internal database for the student's current tasks, upcoming routines, recent health summaries and timetable entries.
3. These structured data points are combined into a context payload and passed to the AI model through a secure API.
4. The AI model generates a response that references real data (for example, number of pending tasks, next exam date, average sleep hours).
5. The response is sent back to the frontend and displayed in the chat interface.

As a result, the assistant can respond with statements such as "There are three urgent tasks due within the next two days" or "Average screen time has been higher than sleep duration this week; consider reducing non-study screen usage in the evening." The assistant also has access to timetable APIs that parse and store class schedules, which can be used to recommend free slots for focused study.

Push notifications

Push notifications are powered by a service worker and web push subscriptions managed through the backend. The VAPID public and private keys are configured in environment variables and used to sign notifications. Examples of notifications include reminders about upcoming routine blocks, approaching deadlines for high-priority tasks and gentle prompts when missions remain incomplete late in the day.

6 Android Deployment

The Android deployment process builds on Capacitor to avoid rewriting the interface as a native app. The steps are:

1. Generate a production build of the React application using:

```
npm run build
```

2. Configure Capacitor in `capacitor.config.ts` with the appropriate application id, name and `webDir` pointing to `build`.
3. Synchronise the web assets with the Android project:

```
npx cap sync android
```

4. Open the Android project in Android Studio using:

```
npx cap open android
```

and run the application on a connected device via USB debugging.

An important configuration detail is the base URL used by the frontend to reach the backend. When the app is loaded on a physical device, `localhost` refers to the device itself, not the development machine. For development builds, the base URL is configured to the local network IP address of the backend host (for example `http://192.168.x.x:5000`). This change resolves apparent “offline” errors when testing on real phones.

7 Testing and Results

Testing covered logical correctness, user flows and perceived usefulness.

Functional testing

Unit-style tests were carried out for functions in the AI logic, scheduling module and health aggregation. Test scenarios included:

- tasks with varying due dates and difficulty to check that urgency and difficulty weights produced expected priority scores,
- days with no tasks, partially completed tasks and fully completed tasks to confirm productivity percentage,
- health logs with different values to validate average computations and quick insight messages.

Integration testing

Integration tests verified that changes made in one section of the app were reflected correctly in others:

- creating and completing tasks updated both the Tasks page and the dashboard cards and charts,
- modifying routines changed the Study Plan summary and the schedule considered by the assistant,
- adding health logs updated the health overview bars and weekly trend charts,
- interacting with the assistant produced responses consistent with the current state of tasks and routines.

The Android deployment was tested on a physical device, confirming that the backend communication worked correctly once the network base URL was configured away from `localhost`.

User feedback

Early users reported that the dashboard reduced confusion about what to work on by highlighting a small set of missions instead of all tasks at once. The health hub helped in visualising how sleep and screen time patterns related to study plans. Students also found the assistant convenient for quickly checking pending work without navigating through multiple pages.

8 Future Enhancements

Several enhancements are planned to extend the Student Productivity Tracker beyond its current capabilities.

Cloud storage and multi-device sync

The existing combination of local storage and a backend can be expanded into full cloud storage with user authentication. This would allow students to access their productivity data and health logs from multiple devices and preserve analytics over longer durations.

Advanced learning-based analytics

The current AI logic uses deterministic rules. Future versions may incorporate learning-based models that adapt parameters, such as urgency weights and recommended study times, based on the individual's history. This would produce more personalised suggestions.

Deeper health–study integration

Stronger integration between the health hub and academic planning is another direction. For example, poor sleep streaks could automatically lower the daily workload or suggest lighter missions until recovery, and consistent hydration and activity could be recognised with positive reinforcement.

Extended assistant capabilities

The assistant can be extended with template queries such as “Generate a study plan for tomorrow” or “Summarise this week's progress in two lines”. Integrations with institutional timetables and exam portals could allow automatic import of deadlines and schedules.

9 Conclusion

The Student Productivity Tracker integrates academic planning, health tracking, analytics and conversational assistance into a single system. Tasks, routines and health logs are treated as first-class data sources, and AI logic converts this information into clear metrics and recommendations presented on an interactive dashboard.

The documentation has described the four core components requested in the assignment: AI logic, routine flow, task scheduling logic and the database schema. Additional sections have covered the health and well-being module, assistant and API integration, software requirements, deployment and testing.

The current implementation already helps students to identify important tasks, visualise study patterns, reflect on health habits and quickly obtain answers about their status from an AI assistant. With future enhancements in cloud storage, advanced analytics and deeper integrations, the system has the potential to develop into a robust academic companion that supports sustainable productivity throughout a student's degree.

Thank You